

RARITONE SECURITY, VALIDATION & MEDIA WORKFLOW DOCUMENTATION

Project Name: Raritone Virtual Try-On Platform

Prepared By: Vagdevi and Nainisha

Module: Security, Validation & Media Workflow Research

Version: 1.0

1. Security Architecture Documentation

The backend implements multiple layers of security to protect APIs, user data, and media resources.

JWT Authentication

Purpose

JWT (JSON Web Token) is used for secure user authentication and authorization.

Authentication Flow

1. User logs in.
2. Server verifies credentials.
3. JWT token is generated.
4. Token is sent to the client.
5. Client includes token in Authorization Header.
6. Middleware verifies token before granting access.

Example Header

Authorization: Bearer <jwt_token>

Benefits

- Stateless authentication
- Secure API access
- Scalable architecture
- Role-based authorization

Joi Validation

Purpose

Validates incoming request data before processing.

Validation Examples

User Registration

```
{
  name: Joi.string().required(),
  email: Joi.string().email().required(),
  password: Joi.string().min(6).required()
}
```

Product Creation

```
{
  name: Joi.string().required(),
  price: Joi.number().required()
}
```

```
}
```

Benefits

- Prevents invalid requests
- Reduces server errors
- Improves API reliability

Helmet Security

Purpose

Helmet secures Express applications by setting HTTP security headers.

Security Headers

- Content-Security-Policy
- X-Frame-Options
- X-Content-Type-Options
- Referrer-Policy

Benefits

- Prevents XSS attacks
- Protects against clickjacking
- Improves browser security

Morgan Logging

Purpose

HTTP request logging middleware.

Logs

- Request method
- Endpoint
- Status code
- Response time

Example

GET /api/products 200 32ms

POST /api/auth/login 401 15ms

Benefits

- API monitoring
- Debugging
- Performance tracking

Rate Limiting

Purpose

Prevents API abuse and brute-force attacks.

Example Configuration

100 requests per 15 minutes

Benefits

- Prevents spam requests
- Reduces DDoS risks

- Protects authentication endpoints

Error Middleware

Purpose

Centralized error handling across the application.

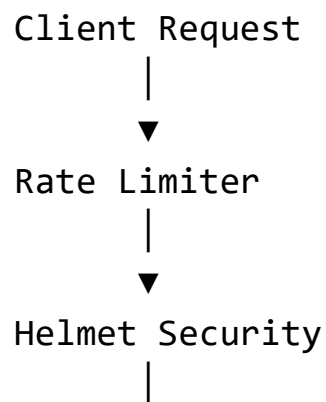
Example Response

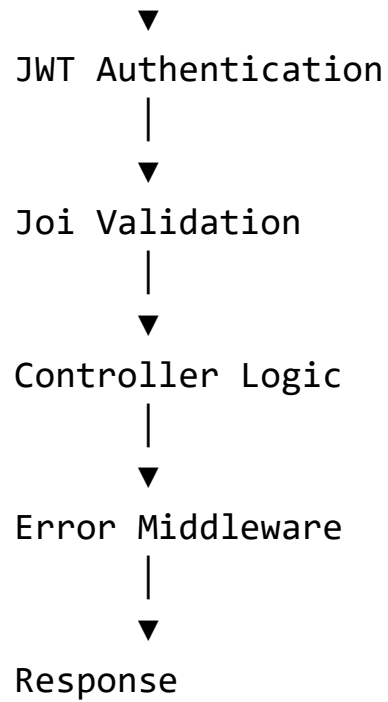
```
{  
  "success": false,  
  "message": "Unauthorized access"  
}
```

Benefits

- Consistent error responses
- Cleaner controller logic
- Easier debugging

Security Flow Diagram





2. Validation Documentation

Validation Layers

Authentication Validation

- Missing Token
- Invalid Token
- Expired Token

Request Validation

- Required Fields
- Data Types
- String Length
- Email Format

Database Validation

- Duplicate Emails
- Invalid Product IDs
- Missing References

Common Validation Responses

Missing Field

```
{  
  "success": false,  
  "message": "\"email\" is required"  
}
```

Invalid Email

```
{  
  "success": false,  
  "message": "\"email\" must be a valid email"  
}
```

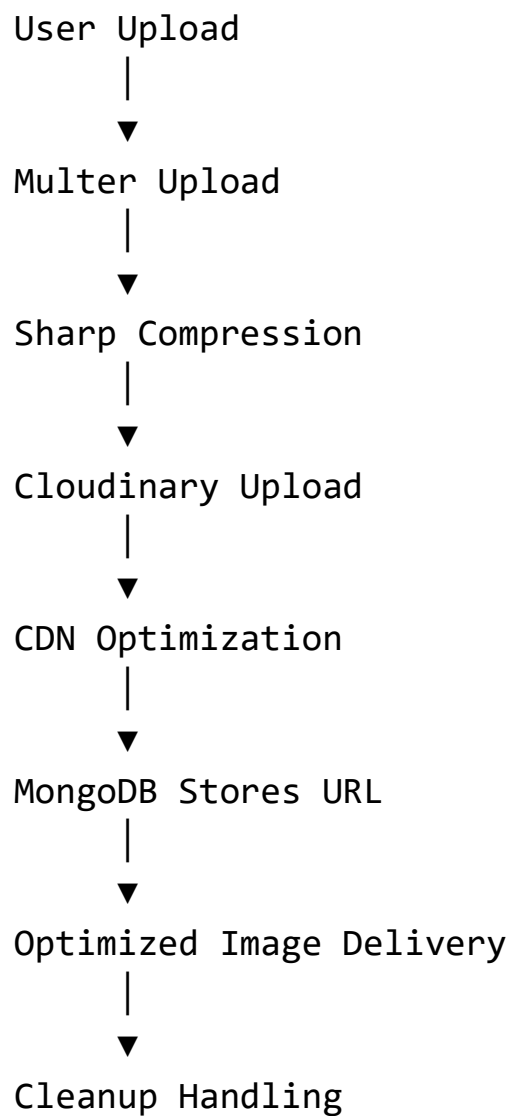
Invalid Product ID

```
{  
  "success": false,  
  "message": "Product not found"  
}
```

3. Media Workflow Documentation

The media pipeline is responsible for uploading, optimizing, storing, and delivering images efficiently.

Media Workflow Architecture



Multer Upload Flow

Purpose

Handles multipart/form-data uploads.

Responsibilities

- File reception
- Temporary storage
- File filtering
- Size validation

Benefits

- Efficient file handling
- Upload restrictions
- Server-side validation

Sharp Compression

Purpose

Image optimization before cloud upload.

Functions

- Resize images
- Compress images
- Convert formats

Benefits

- Reduced file size

- Faster uploads
- Lower storage costs

Cloudinary Upload

Purpose

Stores media assets in the cloud.

Features

- Secure storage
- Automatic transformations
- Global delivery

Stored Assets

- Product Images
- User Avatars
- Try-On Results

CDN Optimization

Purpose

Content Delivery Network serves images globally.

Benefits

- Reduced latency
- Faster loading
- Better user experience

Optimized Image Delivery

Cloudinary Features

- Auto compression
- Auto format selection
- Dynamic resizing

Example

Original: 5 MB

Optimized: 300 KB

Cleanup Handling

Purpose

Remove unnecessary temporary files.

Actions

- Delete local uploads
- Remove failed uploads
- Clean processing files

Benefits

- Reduced disk usage
- Better performance
- Improved stability

4. Security Verification Results

JWT Authentication

- ✓ Login token generation verified
- ✓ Protected routes secured
- ✓ Invalid tokens rejected

Joi Validation

- ✓ Required field validation verified
- ✓ Invalid email detection verified
- ✓ Type validation verified

Helmet Security

- ✓ Security headers active
- ✓ Browser protection enabled

Morgan Logging

- ✓ Request logging verified
- ✓ Error tracking verified

Rate Limiting

- ✓ Excessive requests blocked
- ✓ API abuse protection active

Error Middleware

- ✓ Centralized error responses verified
- ✓ Consistent error format confirmed

5. Postman Testing Documentation

Test Case 1 – Validation Error

Request

POST /api/auth/register

Body:

```
{  
  "name": "Test User"  
}
```

Expected Response

```
{  
  "success": false,  
  "message": "\"email\" is required"
```

```
}
```

Screenshot Required:

Validation Error Response

Test Case 2 – Invalid Token

Request

GET /api/user/profile

Header:

Authorization: Bearer invalid_token

Expected Response

```
{  
  "success": false,  
  "message": "Invalid token"  
}
```

Screenshot Required:

Token Verification Error

Test Case 3 – Unauthorized Access

Request

GET /api/admin/products

Without Token

Expected Response

```
{  
  "success": false,  
  "message": "Access denied"  
}
```

Screenshot Required:

Unauthorized Access Error

Conclusion

The Raritone backend follows a layered security architecture using JWT authentication, Joi validation, Helmet security headers, Morgan logging, rate limiting, and centralized error handling. The media workflow leverages Multer, Sharp, Cloudinary, and CDN optimization to ensure efficient image processing and delivery.

The platform is secure, scalable, and optimized for future AI-powered virtual try-on operations.